

Python Program: How a Neuron Works

Intuition

A neuron is essentially a decision calculator.

Combine inputs using weights

Add bias

Pass through activation

Produce output

Stack millions of these together and you get deep neural networks—and eventually transformers.

Intuition

A neuron is essentially a **decision calculator**.

Combine inputs using weights

Add bias

Pass through activation

Produce output

Stack millions of these together and you get deep neural networks—and eventually transformers.

Import Library

```
import numpy as np
```

NumPy helps perform vector and matrix operations.

Define Inputs

These represent features coming into the neuron.

```
inputs = np.array([0.5, 0.2, 0.1])
```

Example interpretation:

Input Meaning

x1 feature 1
x2 feature 2
x3 feature 3

Define Weights

Each input has a corresponding weight.

```
weights = np.array([0.4, 0.7, 0.2])
```

Weights determine how important each input is.

Define Bias

Bias allows the neuron to shift the output.

```
bias = 0.1
```

Compute Weighted Sum

This is the core neuron calculation.

```
z = np.dot(inputs, weights) + bias
```

```
print("Weighted Sum (z):", z)
```

Formula used:

$$z = x_1w_1 + x_2w_2 + x_3w_3 + b$$

Step 6 — Apply Activation Function

We use **ReLU**.

$$ReLU(x) = \max(0, x)$$

```
def relu(x):  
    return max(0, x)
```

```
output = relu(z)
```

```
print("Neuron Output:", output)
```

Full Program

```
import numpy as np
```

```
# Inputs
```

```

inputs = np.array([0.5, 0.2, 0.1])

# Weights
weights = np.array([0.4, 0.7, 0.2])

# Bias
bias = 0.1

# Weighted sum
z = np.dot(inputs, weights) + bias

print("Weighted Sum:", z)

# Activation function
def relu(x):
    return max(0, x)

output = relu(z)

print("Neuron Output:", output)

```

Example Output
Weighted Sum: 0.41
Neuron Output: 0.41

If the weighted sum were negative, ReLU would output **0**

Concept	Where it appears
Neuron computation	weighted sum
Weights	feature importance
Bias	output adjustment
Activation	ReLU function